# videotest

Учебное пособие: архитектура C++-проекта, обработка видео
и математика компьютерного зрения

Учебные материалы по проекту `videotest`

**Abstract**

Пособие разбирает учебный проект `videotest`: захват видео с камеры через V4L2, инференс ONNX-модели instance-сегментации, сопровождение объектов фильтром Калмана и одновременный показ исходного и размеченного потоков. Документ рассчитан на читателя с сильной математической подготовкой и опытом в нейросетях, но давно не писавшего на C++. Поэтому язык C++ объясняется подробно: владение памятью, времена жизни объектов, многопоточность, система сборки. Отдельные главы посвящены математике проекта: преобразованию координат letterbox, декодированию выходов YOLO, подавлению пересечений (NMS), построению масок из прототипов, фильтру Калмана и метрикам качества.

Всё изложенное соответствует коду в каталоге `src/`; ссылки на файлы и функции даны по ходу текста.

## Contents

# 1 Введение: что делает программа

## 1.1 Постановка задачи

Программа получает поток кадров с веб-камеры (`/dev/videoX`), для каждого кадра запускает нейросетевую модель instance-сегментации, сопоставляет найденные объекты между кадрами (трекинг) и выводит на экран два изображения рядом: слева — исходный кадр, справа — тот же кадр с наложенными масками, рамками и идентификаторами объектов.

| | |
|---|---|
| Источник кадров | V4L2 (Video4Linux2), прямой доступ к драйверу |
| Модель | ONNX Runtime, YOLO-подобная сеть instance-сегментации |
| Трекинг | фильтр Калмана + сопоставление по IoU |
| Вывод | OpenCV HighGUI, два потока рядом |
| Потоки | 2 рабочих + главный (отображение) |

## 1.2 Почему именно такая архитектура

Три ограничения задают всё остальное.

1. **Камера диктует темп.** При 30 кадрах в секунду новый кадр появляется каждые 33,3 мс, и драйвер имеет конечное число внутренних буферов. Если их вовремя не возвращать, камера останавливается.
2. **Инференс непредсказуем по времени.** Один кадр может обрабатываться 8 мс, а может 40 мс — в зависимости от содержимого, загрузки CPU и модели.
3. **Экран должен показывать свежее.** Если обработка отстаёт, разумно пропустить устаревшие кадры, а не выстраивать их в очередь.

Отсюда всё устройство: камера и модель работают в разных потоках, между ними находится очередь *ограниченной ёмкости*, которая при переполнении выбрасывает самый старый кадр. Задержка при этом остаётся постоянной, а не растёт линейно со временем.

## 1.3 Быстрый старт

```
sudo apt install build-essential cmake libopencv-dev v4l-utils   (*@\#@*) OpenCV 4.5+
./scripts/fetch_onnxruntime.sh                                   (*@\#@*) SDK  third_party/
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build --output-on-failure
./build/videotest --model models/yolo11n-seg.onnx --classes 80
```

Без камеры и без весов модели:

```
./scripts/smoke_test.sh
./build/videotest --synthetic --no-window --max-frames 60 \
                  --model build/stub-raw.onnx --classes 3 --snapshot out.png
```

## 1.4 Структура каталогов

```
videotest/
  CMakeLists.txt
  README.md
  docs/
    architecture.md         ()
    tutorial.tex
  scripts/
    fetch_onnxruntime.sh   SDK ONNX Runtime
    make_stub_model.py     -
    smoke_test.sh
  src/
    frame.hpp              ,
    bounded_queue.hpp
    logger.{hpp,cpp}
    stop_flag.{hpp,cpp}      Ctrl+C
    main.cpp               ,
```

```
  streams/
    source.hpp
    v4l2_source.*         V4L2
    synthetic_source.*    ()
  model/
    segmenter.*         ONNX: , ,
    class_names.*         (COCO  )
  track/
    tracker.*             + IoU-
  pipeline.*
  render/
    renderer.*           , , HUD,
tests/
  queue_test.cpp
  tracker_test.cpp
```

Разделение на каталоги отражает *направление зависимостей*: `streams` и `model` не знают друг о друге, `pipeline` знает об обоих, `main` собирает всё вместе. Такое разложение позволяет тестировать трекер вообще без камеры и без модели.

# 2 Как устроен C++-проект и как он собирается

Этот раздел — для читателя, который помнит язык, но забыл инструменты.

## 2.1 Единицы трансляции, заголовки, линковка

Программа на C++ не компилируется целиком. Каждый файл `.cpp` (*единица трансляции*) компилируется независимо в объектный файл `.o`; затем *компоновщик* (linker) соединяет объектные файлы и библиотеки в исполняемый файл.

- **Заголовочный файл** (`.hpp`) содержит *объявления*: типы, сигнатуры функций, шаблоны, встраиваемые (`inline`) функции. Он вставляется в `.cpp` директивой `#include` на этапе препроцессора.
- **Файл реализации** (`.cpp`) содержит *определения*: тела функций, глобальные объекты.
- Директива `#pragma once` (или include guard) защищает от повторного включения одного заголовка в одной единице трансляции.

*Замечание.* Шаблоны классов и функций определены *полностью* в заголовках. Причина: компилятор должен видеть тело шаблона, чтобы сгенерировать код для конкретного типа аргумента. Именно поэтому `Queue<T>` в проекте — один заголовок `bounded_queue.hpp` без `.cpp`.

## 2.2 Статическая и динамическая линковка

| Вид | Что происходит |
|---|---|
| Статическая (`.a`) | Код библиотеки *копируется* в исполняемый файл при компоновке. Файл больше, но зависимостей во время работы нет. |
| Динамическая (`.so`) | В исполняемый файл попадает только имя символа; код подгружается при запуске. Обновление библиотеки не требует пересборки. |

В проекте используется оба вида: собственная библиотека `videotest_core` статическая (её код всё равно нужен целиком), а OpenCV и ONNX Runtime — динамические.

Динамическая библиотека ищется во время выполнения, а не компиляции. Если `libonnxruntime.so` лежит в нестандартном каталоге, программа запустится с ошибкой «cannot open shared object file». Решение — переменная `LD_LIBRARY_PATH` либо *rpath*, вшитый в бинарник при сборке (см. `BUILD_RPATH` в `CMakeLists.txt`).

## 2.3 Система сборки CMake

CMake не компилирует сам: он *генерирует* файлы сборки (Makefile или `build.ninja`) для выбранного генератора. Процесс двухфазный:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release   #@*) конфигурациясmake -build build -j (*@#@*) сборка
```

Ключевые понятия:

- **Цель (target)** — то, что собирается: исполняемый файл, библиотека или тест. Цель хранит свои свойства: исходники, флаги, зависимости.
- **Область видимости** `PRIVATE`/`PUBLIC`/`INTERFACE`. `PUBLIC` означает «мне нужно и всем, кто меня линкует». В проекте `videotest_core` объявляет зависимости (`OpenCV`, `ONNXRUNTIME`) как `PUBLIC` — поэтому `videotest` и тесты получают их автоматически.
- **Тип сборки** (`CMAKE_BUILD_TYPE`) задаёт набор флагов: `Debug` ($-O0\ -g$), `Release` ($-O3\ -DNDEBUG$), `RelWithDebInfo` ($-O2\ -g$).

Минимальный фрагмент из `CMakeLists.txt` проекта:

```
find_package(OpenCV 4.5 REQUIRED COMPONENTS core imgproc imgcodecs highgui dnn video)
find_package(Threads REQUIRED)

add_library(videotest_core STATIC
    src/logger.cpp
    src/pipeline.cpp
    src/track/tracker.cpp
    ...)
target_include_directories(videotest_core PUBLIC src "${ONNXRUNTIME_ROOT}/include")
target_link_libraries(videotest_core PUBLIC
    ${OpenCV_LIBS} Threads::Threads ${ONNXRUNTIME_LIBRARY})

add_executable(videotest src/main.cpp)
target_link_libraries(videotest PRIVATE videotest_core)
```

## 2.4 Флаги компилятора, которые стоит понимать

| Флаг | Смысл |
|---|---|
| `-std=c++17` | версия стандарта языка |
| `-O2`, `-O3` | уровень оптимизации |
| `-g` | отладочная информация (для `gdb`) |
| `-Wall -Wextra` | включить большинство предупреждений |
| `-fsanitize=address` | AddressSanitizer: ловит выход за границы |
| `-fsanitize=undefined` | UBSan: ловит неопределённое поведение |
| `-Werror` | превратить предупреждения в ошибки |

`-fsanitize=address` — самый быстрый способ найти причину `Segmentation fault`. В этом проекте именно так была найдена ошибка индексации в трекере: `gdb` в среде не было, но ASan указал точную строку (`stl_bvector.h:96`, то есть обращение к `std::vector<bool>` за границей).

## 2.5 Тестирование через CTest

`enable_testing()` в CMake и `add_test(NAME ... COMMAND ...)` регистрируют тесты; запуск — `ctest -output-on-failure`. Проект сознательно обходится без фреймворков: тесты — это обычные программы, возвращающие ненулевой код при ошибке.

```
#@*)define CHECK(cond) (*@\@*)do  (*@\@*)if (!(cond))  (*@\@*)std::printf(" FAIL
    ++g_failed; (*@\@*)(*@\@*)while(0)
```

Макрос `#cond` — приём препроцессора: аргумент превращается в строку, поэтому в отчёте видно само условие, а не только номер строки.

### 2.5.1 Главное правило: тест должен падать на ошибке

Бесполезный тест — тот, который проходит всегда. Поэтому после написания теста его проверяют *обратным* экспериментом: временно возвращают ошибочный код и убеждаются, что тест краснеет.

В проекте так проверены оба новых теста:

# 3 Язык C++, который используется в проекте

## 3.1 Управление памятью: стек, куча, времена жизни

В C++ нет сборщика мусора. Объект, созданный в автоматической памяти (на стеке), уничтожается при выходе из области видимости; объект в динамической памяти (в куче, `new`) живёт до явного `delete`.

```
void f() {
    std::string s = "";       // :
    auto* p = new std::string(""); // :   delete p
    delete p;                          //
}   // s
```

Ручное `new`/`delete` в современном C++ почти не применяется: его заменяют *умные указатели* и принцип RAII.

## 3.2 RAII: ресурс — это объект

**Определение 3.1. RAII** (Resource Acquisition Is Initialization) — приём, при котором владение ресурсом (память, файл, блокировка, сокет) привязано к времени жизни объекта: ресурс захватывается в конструкторе и освобождается в деструкторе.

Пример из проекта — деструктор источника видео:

```
V4l2Source::~V4l2Source() {
    close();          // STREAMOFF -> munmap -> ::close
}
```

Порядок здесь не случаен: нельзя размечать (mmap) память уже закрытого дескриптора. RAII делает этот порядок единственно возможным.

## 3.3 Умные указатели

| Тип | Владение | Когда применяется |
|---|---|---|
| `std::unique_ptr<T>` | единоличное | ресурс принадлежит одному объекту |
| `std::shared_ptr<T>` | разделяемое, со счётчиком ссылок | данные нужны нескольким владельцам |
| `std::weak_ptr<T>` | не владеет | наблюдение без продления жизни |

### 3.3.1 Почему в конвейере именно `shared_ptr`

Кадр нужен одновременно в двух местах: он лежит в очереди, и на него же ссылается результат обработки. При этом заранее неизвестно, кто отпустит его последним.

```
struct Frame {
    cv::Mat image;
    std::uint64_t id = 0;
    std::chrono::steady_clock::time_point captured;
};
struct Result {
    std::shared_ptr<const Frame> frame;   //  ,
    std::vector<Instance> instances;
};

using FramePtr  = std::shared_ptr<Frame>;
using ResultPtr = std::shared_ptr<Result>;
```

Копия `shared_ptr` увеличивает счётчик ссылок атомарно; уничтожение последней копии освобождает объект. Пиксели при этом *не копируются*: `cv::Mat` внутри `Frame` сам является дешёвым дескриптором с разделяемым буфером.

#### 3.3.2 Почему `shared_ptr<const Frame>`

Это ключевой приём всей потокобезопасности проекта. Тип `const Frame` делает кадр *неизменяемым* для всех, кто держит указатель. Пока отрисовывающий поток читает пиксели, обрабатывающий поток уже занят следующим кадром — но записать в этот кадр он не может, потому что `const`. Гонки данных нет *без единого мьютекса* вокруг самих данных.

`const` в C++ защищает от записи через данный указатель, но не гарантирует неизменность объекта вообще. Здесь этого достаточно, потому что никто больше не имеет неконстантного указателя на тот же `Frame`.

### 3.4 Ссылки, копирование, перемещение

- `const T&` — ссылка без копирования, без права изменять. Основной способ передавать большие объекты.
- `T&` — ссылка с правом изменять (выходной параметр).
- `std::move(x)` — не перемещает сам по себе, а *разрешает* вызвать перемещающий конструктор: содержимое объекта переходит в новый, исходный остаётся в допустимом, но неопределённом состоянии.

```
//    :
bool grab(FramePtr& out, std::string& error);
```

Такой стиль (функция возвращает `bool`, результат — через ссылку, текст ошибки — тоже через ссылку) выбран потому, что он не требует исключений на границе потоков и хорошо читается в циклах.

### 3.5 Классы, виртуальные функции, интерфейсы

Абстрактный класс `Source` задаёт контракт:

```
class Source {
public:
    virtual ~Source() = default;
    virtual bool open(std::string& error) = 0;
    virtual bool grab(FramePtr& out, std::string& error) = 0;
    virtual int width() const = 0;
    virtual int height() const = 0;
};
```

- `= 0` делает функцию *чисто виртуальной*: класс нельзя создать, только унаследовать от него.
- `virtual ~Source() = default` обязателен: удаление объекта производного класса через указатель на базовый без виртуального деструктора — неопределённое поведение.
- Вызов виртуальной функции идёт через таблицу виртуальных функций (vtable), то есть стоит одного косвенного перехода. Для вызова раз в кадр это несущественно.

### 3.6 Шаблоны

Очередь — шаблон класса, потому что одна и та же логика нужна для `FramePtr` и `ResultPtr`:

```
template <class T>
class Queue {
public:
    std::size_t push(T value);
    template <class Rep, class Period>
    std::optional<T> pop(std::chrono::duration<Rep, Period> timeout);
};
```

Второй шаблон (шаблонный метод) нужен, чтобы принимать любой тип длительности: `100ms`, `std::chrono::milliseconds(10)` и т.п.

### 3.7 std::optional: отсутствие значения как тип

```
std::optional<T> pop(...);   //  ,
```

Возвращать «магическое» значение (например, пустой указатель) можно, но тип не говорит об этом вызывающему. `std::optional` делает отсутствие значения частью типа, и компилятор заставляет его обработать.

### 3.8 Идиома pimpl, или сокрытие зависимости

Заголовок `segmenter.hpp` не включает `onnxruntime_cxx_api.h`: этот заголовок тяжёлый и содержит макросы, которые не должны протекать во все единицы трансляции. Вместо этого объявлена неполная структура:

```
struct OrtHolder;          //
OrtHolder* ort_ = nullptr; //
```

Определение живёт в `.cpp`:

```
struct Segmenter::OrtHolder {
    Ort::Env env{ORT_LOGGING_LEVEL_ERROR, "videotest"};
    Ort::SessionOptions options;
    std::unique_ptr<Ort::Session> session;
    Ort::MemoryInfo memory{"Cpu", OrtArenaAllocator, 0, OrtMemTypeDefault};
};
```

Здесь ручное `new`/`delete` (в деструкторе `Segmenter`). Это осознанный компромисс. С `std::unique_ptr<OrtHolder>` требуется полное определение типа там, где генерируется код уничтожения, то есть в `.cpp`. Технически это решается объявлением деструктора в заголовке и его определением в `.cpp` (что и сделано), но тогда `unique_ptr` не даёт выигрыша в читаемости: деструктор всё равно написан вручную. Вариант с указателем явнее показывает, что владение единоличное и освобождается в одном месте.

### 3.9 Обработка ошибок и границы потоков

В C++ исключение, покинувшее поток, вызывает `std::terminate()`. Поэтому на границе рабочего потока исключения *переводятся в код возврата*:

```
try {
    ... session->Run(...);
} catch (const Ort::Exception& e) {
    error = std::string("ONNX Runtime: ") + e.what();
    return false;      //     std::thread
}
```

## 4 Данные, ходящие между потоками

### 4.1 Три структуры и их инварианты

| Тип | Инварианты |
|---|---|
| `Frame` | `image` — BGR, `CV_8UC3`; `captured` — момент получения от драйвера, монотонные часы. |
| `Instance` | `box` в целых пикселях исходного кадра; `mask` либо пустая, либо `CV_8UC1` размера кадра со значениями 0/255; `track_id = -1` до первого присвоения. |
| `Result` | `frame` — неизменяемый указатель на кадр; вектор `instances` отсортирован по убыванию уверенности. |

Почему `steady_clock`, а не `system_clock`: последний может быть переведён назад (синхронизация времени, переход на летнее время), и тогда «задержка» окажется отрицательной. `steady_clock` гарантированно монотонен.

*Замечание.* Почему координаты целые: рисование всё равно округляет, а целочисленные операции дешевле. `cv::Rect2f` применяется только внутри преобразований, где важна дробная точность.

# 5 Очередь с вытеснением: механизм и обоснование

## 5.1 Постановка задачи

Нужен буфер между производителем (камера, 30 кадров/с, ждать нельзя) и потребителем (модель, время обработки переменное). Требования:

1. производитель никогда не блокируется;
2. потребитель получает *свежие* данные;
3. задержка «попадания в обработку» ограничена константой;
4. остановка программы не должна зависать на ожидании данных.

Классическая очередь FIFO без ограничения размера нарушает пункты 2 и 3: если модель медленнее камеры, длина очереди растёт, и обрабатывается кадр десятисекундной давности.

## 5.2 Политика: вытеснение самого старого

**Определение 5.1** (Очередь с вытеснением)**.** Очередь ёмкости $C$: операция PUSH всегда завершается успешно. Если очередь заполнена, предварительно удаляется элемент, поступивший раньше всех. Операция POP возвращает самый старый элемент либо, при таймауте, ничего.

Точное соотношение баланса: в любой момент

$$N = M + D + |Q|,$$

где $N$ — сколько раз вызван PUSH, $M$ — сколько раз успешно вызван POP, $D$ — сколько элементов вытеснено (это число возвращает PUSH), $|Q|$ — текущее содержимое очереди. После того как производитель закончил и очередь опустела, $|Q| = 0$ и

$$D = N - M.$$

Именно это тождество проверяет тест `queue_tests`.

## 5.3 Реализация

```
template <class T>
class Queue {
public:
    explicit Queue(std::size_t capacity)
        : capacity_(capacity == 0 ? 1 : capacity) {}

    std::size_t push(T value) {
        std::size_t dropped = 0;
        {
            std::lock_guard lock(mutex_);
            if (items_.size() >= capacity_) {
                items_.pop_front();
                ++dropped;
            }
            items_.push_back(std::move(value));
        }
        not_empty_.notify_one();
        return dropped;
    }

    template <class Rep, class Period>
    std::optional<T> pop(std::chrono::duration<Rep, Period> timeout) {
        std::unique_lock lock(mutex_);
        not_empty_.wait_for(lock, timeout, [this] { return !items_.empty(); });
        if (items_.empty()) return std::nullopt;
        T value = std::move(items_.front());
        items_.pop_front();
        return value;
```

```
    }
private:
    mutable std::mutex mutex_;
    std::condition_variable not_empty_;
    std::deque<T> items_;
    const std::size_t capacity_;
};
```

Разберём каждое решение.

### 5.3.1 Почему `std::deque`, а не `std::vector`

Вытеснение удаляет элемент из *начала*, добавление идёт в *конец*. `std::deque` поддерживает обе операции за $O(1)$; `std::vector` удаление из начала делает за $O(n)$. При ёмкости 2–4 это неважно по абсолютному времени, но выбор структуры должен соответствовать операциям.

### 5.3.2 Почему `std::mutex` и `condition_variable`

Мьютекс защищает инвариант «`items_` согласован». Условная переменная позволяет потребителю *спать*, а не крутиться в цикле, расходуя ядро.

### 5.3.3 Почему `unique_lock`, а не `lock_guard`

`wait_for` должен *отпустить* мьютекс на время сна и снова взять его после пробуждения. `std::unique_lock` это умеет, `std::lock_guard` — нет.

### 5.3.4 Почему предикат обязателен

```
not_empty_.wait_for(lock, timeout, [this] { return !items_.empty(); });
```

Условная переменная может проснуться без уведомления (*ложное пробуждение*) и может быть разбужена не тем событием. Без предиката код продолжил бы работу с пустой очередью. С предикатом `wait_for` возвращается только когда условие истинно либо истёк таймаут.

### 5.3.5 Почему уведомление вне блокировки

```
{
    std::lock_guard lock(mutex_);
    ...
}                 //
not_empty_.notify_one();   //
```

Если уведомить под мьютексом, разбуженный поток сразу же упрётся в заблокированный мьютекс (*thundering herd* / лишнее переключение). Уведомление после снятия блокировки этого избегает.

### 5.3.6 Почему таймаут в `pop`

Без таймаута поток-обработчик, ожидая кадр, никогда не узнал бы о том, что программу просят остановить. Таймаут 100 мс означает: «даже если данных нет, проснись и проверь флаг остановки».

## 5.4 Ёмкость 2: почему именно столько

- Ёмкость 1: пока потребитель обрабатывает кадр, производитель уже положил следующий — один кадр гарантированно ждёт. Любая задержка сразу приводит к вытеснению.
- Ёмкость 2: допускает кратковременную неравномерность (один кадр в работе, один в запасе), при этом задержка ограничена сверху временем двух кадров.
- Ёмкость $\gg 2$: задержка растёт пропорционально, смысл вытеснения теряется.

Число вытеснений выведено в HUD как `dropped`. Это прямой индикатор: если счётчик растёт, модель не успевает за камерой, и на экране видно меньше кадров, чем снимает камера. Именно так и должно быть — обработка не тормозит приём.

# 6 Потоки и синхронизация

## 6.1 Схема

| Поток | Что делает | Куда пишет |
|---|---|---|
| 1. Захват | `source.grab()` в цикле | `Queue<FramePtr>`(2) |
| 2. Обработка | `pop()` → `infer()` → `tracker.update()` | `Queue<ResultPtr>`(2) |
| Главный | `pop(10 ms)` → отрисовка → `imshow` | экран |

## 6.2 Порядок остановки — источник тонкой ошибки

```
void Pipeline::stop() {
    running_.store(false, std::memory_order_relaxed);  // 1.
    if (capture_thread_.joinable()) capture_thread_.join();   // 2.
    if (process_thread_.joinable()) process_thread_.join();
}
```

Если поменять местами, получится *дедлок*: `join` ждёт завершения потока, а поток крутится по флагу, который ещё не снят. В первой версии эта ошибка присутствовала и проявлялась как `terminate called without an active exception`.

Почему `terminate`: `std::thread`, который остаётся *joinable* при разрушении, вызывает `std::terminate`. Разрушение происходило по выходу из `main` при уже завершившихся циклах.

## 6.3 Атомарные счётчики

Метрики (получено, обработано, вытеснено) читаются главным потоком и пишутся рабочими. Здесь достаточно `std::atomic<std::uint64_t>`: операция `fetch_add` не требует блокировки. Это дешевле мьютекса и не создаёт конкуренции на кадр.

## 6.4 Остановка по Ctrl+C

Обработчик сигнала обязан быть *асинхронно-безопасным*: в нём нельзя брать мьютексы, вызывать `printf`, выделять память. Единственное допустимое действие — записать в атомарный флаг:

```
extern "C" void on_signal(int) {
    stop_flag().store(true, std::memory_order_relaxed);
}
```

Рабочие циклы сами проверяют флаг: `poll()` возвращается по таймауту 500 мс, `Queue::pop()` — по таймауту 100 мс.

# 7 Захват видео через V4L2

## 7.1 Что такое V4L2

Video4Linux2 — набор системных вызовов `ioctl` для работы с видеоустройствами в Linux. Камера представляется файлом `/dev/videoX`; управление идёт командами `VIDIOC_*`.

## 7.2 Последовательность вызовов

```
open(/dev/videoX, O_RDWR | O_NONBLOCK)
  VIDIOC_QUERYCAP
  VIDIOC_S_FMT
  VIDIOC_S_PARM
  VIDIOC_REQBUFS         N   (MMAP)
  VIDIOC_QUERYBUF + mmap
  VIDIOC_QBUF x N
  VIDIOC_STREAMON
: poll() -> VIDIOC_DQBUF ->  -> VIDIOC_QBUF
```

### 7.3 Тонкости, без которых не работает

1. **`device_caps`, а не `capabilities`.** У камеры с микрофоном флаг `capabilities` описывает всё устройство. Реальные возможности конкретного узла лежат в `device_caps`, и проверка только по `capabilities` ложно отвергает часть камер.
2. **`poll` перед `DQBUF`.** Файл открыт с `O_NONBLOCK`, поэтому `DQBUF` без данных вернул бы `EAGAIN`, и пришлось бы крутиться в busy-loop, съедая ядро.
3. **`QBUF` обязателен всегда.** После `DQBUF` буфер принадлежит программе. Не вернув его, через `buffer_count` кадров драйверу нечего заполнять, и камера встанет.
4. **`bytesperline` (шаг строки).** Длина строки в памяти может быть больше width × bpp из-за выравнивания. Матрица создаётся с явным шагом (`cv::Mat(height, width, CV_8UC2, data, step)`); без него картинка «поедет» по диагонали.
5. **`EINTR`.** Системный вызов может быть прерван сигналом (например, нашим SIGINT). Это не ошибка устройства, поэтому `ioctl` повторяется (в проекте — шаблонная обёртка `xioctl`, принимающая переменное число аргументов):

```
template <class... Args>
int xioctl(int fd, unsigned long request, Args... args) {
    int result;
    do {
        result = ::ioctl(fd, request, args...);
    } while (result == -1 && errno == EINTR);
    return result;
}
```

### 7.4 Почему mmap, а не read

При `read()` драйвер копирует кадр в буфер пользователя при каждом вызове. При mmap память драйвера отображается в адресное пространство процесса: копия остаётся только при конвертации формата (её делает OpenCV). На 1280×720 и 30 кадрах в секунду это заметная разница.

### 7.5 MJPG и YUYV

| Формат | Хранение | Следствия |
|---|---|---|
| MJPG | каждый кадр — JPEG | меньше по шине USB, но требует декодирования (единицы мс); нет потерь в `Mat`, потери в JPEG |
| YUYV | 2 байта на пиксель, подвыборка цветности | не требует декодирования, но по USB 2.0 не всегда влезает 1280×720 при 30 кадрах/с |

*Замечание.* Формат в проекте задаётся *явно* (`-format MJPG|YUYV`). Автоподбор убран сознательно: он скрывает, что реально поддерживает устройство. Если драйвер подменил формат, программа выдаёт ошибку с подсказкой `v4l2-ctl -list-formats-ext`.

## 8 Модель: от тензора к объектам

### 8.1 Препроцессинг: letterbox

Сеть ждёт квадратный вход $S \times S$ (обычно $S = 640$), значения $\mathbb{R}^{3\times S\times S}$ в диапазоне $[0, 1]$, порядок каналов RGB. Кадр имеет размер $W \times H$.

#### 8.1.1 Математика

Вычисляется масштаб

$$s = \min\left(\frac{S}{W}, \frac{S}{H}\right),$$

новый размер $W' = \lfloor sW \rceil$, $H' = \lfloor sH \rceil$, и отступы

$$p_x = \frac{S - W'}{2}, \qquad p_y = \frac{S - H'}{2}.$$

**Определение 8.1** (Прямое преобразование)**.** Пиксель кадра $(u, v)$ отображается в точку сети

$$x = su + p_x, \qquad y = sv + p_y.$$

**Определение 8.2** (Обратное преобразование)**.** По координатам сети $(x, y)$ координата кадра восстанавливается как

$$u = \frac{x - p_x}{s}, \qquad v = \frac{y - p_y}{s}.$$

Пара $(s, p_x, p_y)$ — единственное, что нужно сохранить при прямом ходе, чтобы правильно вернуть координаты обратно. Именно поэтому `preprocess()` возвращает `scale` и `pad` по ссылке.

Почему нельзя просто `resize` в квадрат: при разных $W$ и $H$ это даёт *разные* масштабы по осям, и обратное преобразование $x$ и $y$ должно выполняться с разными коэффициентами. Забыть об одном из них — классическая ошибка, дающая сдвиг, растущий к краю кадра.
Отступы заполняются значением 114: это стандартное значение YOLO при обучении, и сеть к нему привыкла.

### 8.1.2 Приведение к входу сети

$$\text{blob}[c][y][x] = \frac{\text{canvas}[y][x][c']}{255}, \qquad c' = \begin{cases} 2 - c, & \text{для RGB (swapRB)}, \\ c, & \text{иначе}, \end{cases} \tag{1}$$

где $c \in \{0, 1, 2\}$ — индекс канала в NCHW. Функция `cv::dnn::blobFromImage` выполняет и перестановку осей HWC→NCHW, и деление на 255, и перевод BGR→RGB.

## 8.2 Выходы модели

### 8.2.1 Формат `raw` (YOLOv8-seg, YOLO11-seg)

$$\text{output0} \in \mathbb{R}^{1 \times (4 + n_c + 32) \times N}, \qquad \text{output1} \in \mathbb{R}^{1 \times 32 \times h \times w},$$

где $n_c$ — число классов, $N$ — число якорей (8400 при $S = 640$), $h, w$ — размер карты прототипов (160 при $S = 640$).
Раскладка *канал-мажорная*: значение канала $c$ для якоря $a$ лежит по индексу

$$\text{idx}(c, a) = c \cdot N + a.$$

Столбцы: $c_x, c_y, w, h$, затем $n_c$ вероятностей классов, затем 32 коэффициента масок.

### 8.2.2 Формат `nms` (YOLO26, end-to-end)

$$\text{output0} \in \mathbb{R}^{1 \times M \times (6 + 32)}, \qquad M = 300,$$

столбцы: $x_1, y_1, x_2, y_2$, уверенность, номер класса, 32 коэффициента. Подавление пересечений уже выполнено внутри графа.

Повторно применять NMS к выходу `nms` нельзя: это удалит легитимно перекрывающиеся объекты, которые сеть намеренно оставила.

## 8.3 Декодирование и уверенность

Для каждого якоря $a$ уверенность класса определяется как максимум по классам:

$$(\hat{s}, \hat{k}) = \big(\max_c p_{c,a},\ \arg\max_c p_{c,a}\big).$$

Якорь отбрасывается, если $\hat{s} < \theta_{\text{score}}$ (порог 0.35 по умолчанию) либо если ширина/высота неположительны.

## 8.4 Подавление пересечений (NMS)

### 8.4.1 Метрика IoU

**Определение 8.3** (IoU, Intersection over Union)**.** Для прямоугольников $A$ и $B$

$$\mathrm{IoU}(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}.$$

Для прямоугольников, выровненных по осям, площади вычисляются через пересечение проекций:

$$|A \cap B| = \max(0, x_2^{\cap} - x_1^{\cap}) \cdot \max(0, y_2^{\cap} - y_1^{\cap}),$$

$$x_1^{\cap} = \max(x_1^A, x_1^B), \quad x_2^{\cap} = \min(x_2^A, x_2^B),$$

и аналогично по $y$. Знаменатель нулевым быть не может (площади положительны), но в коде предусмотрена защита от деления на ноль для вырожденных боксов.

### 8.4.2 Алгоритм

1. Отсортировать детекции по убыванию уверенности.
2. Взять лучшую непогашенную детекцию, оставить её.
3. Погасить все следующие детекции *того же класса* с $\mathrm{IoU} > \theta_{\mathrm{nms}}$ (0.45 по умолчанию).
4. Повторять, пока есть непогашенные.

Подавление *class-aware* принципиально: человек и чашка часто перекрываются. Подавление по всем классам склеило бы их в один объект. Сложность алгоритма — $O(n^2)$ по числу детекций после порога уверенности; на практике $n$ невелико (десятки).

## 8.5 Маски: линейная комбинация прототипов

Сеть не выдаёт маску на объект. Она выдаёт:

- базис из 32 «мягких» масок (прототипов) $P_k \in \mathbb{R}^{h \times w}$, $k = 1..32$;
- для каждого объекта — 32 коэффициента $e_k$.

Маска объекта определяется как

$$M(x,y) = \sigma\Big(\sum_{k=1}^{32} e_k \, P_k(y,x)\Big), \qquad \sigma(z) = \frac{1}{1+e^{-z}}.$$

### 8.5.1 Порог без вычисления сигмоиды

Условие $\sigma(z) > t$ эквивалентно

$$z > \ln \frac{t}{1-t} =: \mathrm{logit}(t).$$

При $t = 0.5$ порог равен нулю. Это позволяет заменить $h \cdot w$ вычислений экспоненты одним сравнением:

```
const float logit = std::log(config_.mask_threshold / (1.0f - config_.mask_threshold));
cv::compare(resized, logit, binary, cv::CMP_GT);
```

### 8.5.2 Приведение маски к кадру

Прототипы имеют размер $h \times w$ (160), что соответствует входу сети $S \times S$. Бокс объекта задан в координатах сети. Поэтому:

1. линейная комбинация даёт маску $h \times w$;
2. обрезаем её по боксу в масштабе прототипов: коэффициенты $\rho_x = w/S$, $\rho_y = h/S$;
3. масштабируем обрезанное к размеру бокса *в пикселях кадра*;
4. сравниваем с логит-порогом;
5. вставляем в пустое изображение размера кадра.

# 9 Трекинг: фильтр Калмана и сопоставление

## 9.1 Зачем нужен трекер

Детектор работает независимо на каждом кадре:

- рамка «дрожит» из-за шума сети;
- на части кадров объект не детектируется вовсе (перекрытие, размытие);
- нет понятия «тот же объект» — только множество рамок.

Трекер решает три задачи: сглаживание, предсказание при пропуске детекции и присвоение устойчивого идентификатора.

## 9.2 Модель состояния

**Определение 9.1.** Вектор состояния трека

$$\mathbf{x} = (c_x, c_y, w, h, v_x, v_y, v_w, v_h)^\top \in \mathbb{R}^8,$$

где первые четыре компоненты — центр и размеры рамки, последние четыре — их скорости.

**Определение 9.2** (Модель равномерного движения)**.** Матрица перехода

$$F = \begin{pmatrix} I_4 & I_4 \\ 0 & I_4 \end{pmatrix} \in \mathbb{R}^{8\times 8},$$

то есть $\mathbf{x}_{t+1} = F\mathbf{x}_t$ даёт $c_x \leftarrow c_x + v_x$ и т.д.

**Определение 9.3** (Модель измерений)**.** Детектор сообщает только положение и размер, поэтому

$$H = \begin{pmatrix} I_4 & 0 \end{pmatrix} \in \mathbb{R}^{4\times 8}, \qquad \mathbf{z}_t = H\mathbf{x}_t + \mathbf{v}_t.$$

## 9.3 Уравнения фильтра

Фильтр Калмана поддерживает оценку $\hat{\mathbf{x}}$ и её ковариацию $P$.

**Предсказание.**

$$\hat{\mathbf{x}}_t^- = F\hat{\mathbf{x}}_{t-1}, \tag{2}$$

$$P_t^- = FP_{t-1}F^\top + Q. \tag{3}$$

**Коррекция по измерению $\mathbf{z}_t$.**

$$K_t = P_t^- H^\top \big(HP_t^- H^\top + R\big)^{-1}, \tag{4}$$

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_t^- + K_t\big(\mathbf{z}_t - H\hat{\mathbf{x}}_t^-\big), \tag{5}$$

$$P_t = (I - K_t H)P_t^-. \tag{6}$$

Здесь $Q$ — ковариация шума процесса, $R$ — ковариация шума измерений. В проекте обе заданы диагональными:

$$Q = 10^{-2} I_8, \qquad R = 10^{-1} I_4, \qquad P_0 = I_8.$$

## 9.4 Смысл матрицы усиления

Матрица $K_t$ определяет, насколько доверять новому измерению. Если $P^- \gg R$ (неопределённость велика, измерения точны), то $K \to I$ и оценка почти равна измерению. Если $P^- \ll R$ (оценка уверенна, измерения шумны), $K \to 0$ и оценка почти не меняется.

**Почему нельзя заменить предсказание на $Fx$.** Вторая строка, $P^- = FPF^\top + Q$, обязательна. Она *увеличивает* неопределённость перед новым измерением. Без неё $P$ не растёт, $K$ остаётся крошечным, и коррекция почти не сдвигает состояние: трек «залипает» на начальном положении. В проекте эта ошибка присутствовала и проявлялась так: объект уезжал, IoU с его рамкой падал ниже порога, трек терялся, и идентификатор пересоздавался почти каждый кадр.

Практический вывод, зафиксированный тестом `tracker_tests`: объект, смещающийся на 15 пикселей за кадр, обязан сохранить свой идентификатор на протяжении 25 кадров. Без вызова `cv::KalmanFilter::predict()` тест падает.

### 9.5 Сопоставление детекций и треков

1. Для каждой пары (детекция, трек) вычисляется IoU.
2. Пары с IoU $\geq 0.3$ сортируются по убыванию.
3. Жадно выбираются пары без повторов: одна детекция — один трек.
4. Сопоставленные треки корректируются измерением, получают прежний идентификатор.
5. Несопоставленные детекции создают новые треки с новым идентификатором.
6. Треки без детекции увеличивают счётчик пропусков; при превышении `max_age` $= 15$ удаляются.

Жадное сопоставление — упрощение по сравнению с венгерским алгоритмом (оптимальным по суммарному IoU). На 10–50 объектах разница в качестве мала, а код проще на порядок. Замена локализована в одной функции.

### 9.6 Обоснование выбора порогов

- IoU $\geq 0.3$: рамки соседних кадров при 30 кадрах/с перекрываются сильнее; порог 0.3 отсекает «соседние» объекты, но терпит дрожание.
- `max_age` $= 15$: полсекунды при 30 кадрах/с. Достаточно, чтобы пережить кратковременное перекрытие, но не настолько, чтобы «держать» ушедший объект.

## 10 Метрики качества

### 10.1 Метрики детекции

#### 10.1.1 Точность, полнота, $F_1$

Для детекций вводится понятие совпадения: детекция засчитывается как верная (TP), если её IoU с размеченной рамкой превышает порог (обычно 0.5) и класс совпадает.

| Величина | Определение | Смысл |
|---|---|---|
| Precision | $\dfrac{TP}{TP+FP}$ | доля верных среди предсказанных |
| Recall | $\dfrac{TP}{TP+FN}$ | доля найденных среди размеченных |
| $F_1$ | $\dfrac{2PR}{P+R}$ | гармоническое среднее |

#### 10.1.2 AP и mAP

Кривая *precision–recall* строится варьированием порога уверенности. Average Precision — площадь под ней:

$$\mathrm{AP} = \int_0^1 p(r)\,dr.$$

На практике интеграл заменяется суммой по точкам интерполяции:

$$\mathrm{AP} \approx \sum_k (r_{k+1} - r_k)\,\tilde{p}(r_{k+1}),$$

где $\tilde{p}(r) = \max_{r' \geq r} p(r')$ — монотонная огибающая.

**Определение 10.1** (mAP)**.**

$$\mathrm{mAP} = \frac{1}{n_c} \sum_{c=1}^{n_c} \mathrm{AP}_c,$$

среднее AP по классам. Порог IoU фиксирует вариант метрики (`mAP@0.5`, `mAP@0.5:0.95` — усреднение по порогам 0.5, 0.55, …, 0.95).

### 10.1.3 Метрики сегментации

Для масок IoU считается по пикселям:

$$\mathrm{IoU}_{\mathrm{mask}}(A, B) = \frac{|\{p : A(p) = 1 \wedge B(p) = 1\}|}{|\{p : A(p) = 1 \vee B(p) = 1\}|}.$$

- **Dice** (коэффициент Сёренсена): $\mathrm{Dice} = \dfrac{2|A \cap B|}{|A| + |B|}$, связано с IoU как $\mathrm{Dice} = \dfrac{2\,\mathrm{IoU}}{1 + \mathrm{IoU}}$.
- **mIoU** — среднее IoU по классам (semantic segmentation).
- **Mask mAP** — AP, где совпадение определяется по IoU масок.

## 10.2 Метрики трекинга

### 10.2.1 MOTA

$$\mathrm{MOTA} = 1 - \frac{\sum_t \left(FN_t + FP_t + IDSW_t\right)}{\sum_t GT_t},$$

где $FN$ — пропущенные, $FP$ — ложные, $IDSW$ — переключения идентификатора, $GT$ — число размеченных объектов. Значение может быть отрицательным при большом числе ошибок.

### 10.2.2 IDF1 и MOTP

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}},$$

$$\mathrm{MOTP} = \frac{\sum_{i \in \mathrm{TP}} \mathrm{IoU}_i}{|\mathrm{TP}|}$$

— среднее совпадение для верно сопоставленных объектов (мера точности локализации).

## 10.3 Метрики реального времени

| Метрика | Определение |
|---|---|
| Задержка (latency) | $t_{\text{разметка}} - t_{\text{получение кадра}}$ |
| Пропускная способность | число кадров, обработанных в секунду |
| Доля вытеснений | $\dfrac{D}{N}$ — показывает, насколько модель отстаёт |

Ключевое свойство правильной схемы: при перегрузке *средняя задержка не растёт.* В проекте измерялось 640 × 480 с моделью-заглушкой: спокойный режим (30 кадров/с) — 42 мс, перегрузка (200 кадров/с) — 23 мс. В перегрузке задержка *меньше*, потому что конвейер не ждёт следующего кадра камеры: он обрабатывает непрерывно, а устаревшее вытесняется.

Числа задержки включают путь «получено от драйвера → разметка готова». В спокойном режиме значительная их часть — это ожидание, пока кадр полежит в очереди: он приходит раз в 33 мс, а обрабатывается за 10 мс.

# 11 Декодирование и отрисовка: практические детали

## 11.1 Почему маски рисуются одним слоем

Если накладывать каждую маску отдельно через `addWeighted`, в местах перекрытия альфа складывается:

$$C_{\text{out}} = \alpha C_1 + (1-\alpha)\big(\alpha C_2 + (1-\alpha)C_0\big) \neq \alpha C_1 + (1-\alpha)C_0,$$

и пятно темнеет. В проекте маски красятся в отдельный слой, который смешивается с кадром *один* раз.

## 11.2 Почему `findContours` на копии

`cv::findContours` модифицирует входную матрицу (историческая особенность OpenCV). Поскольку `Instance.mask` может читаться другим потоком, контуры строятся по копии ROI:

```
cv::findContours(instance.mask(roi).clone(), contours, cv::RETR_EXTERNAL,
                 cv::CHAIN_APPROX_SIMPLE);
```

## 11.3 Принцип «нейтральной разметки»

Если модели нет, вектор `instances` пуст, и функция `draw_overlay` возвращает обычную копию кадра. Правая половина склейки оказывается побитово равна левой. Отдельного режима «без модели» нет — это важно: конвейер всегда работает одинаково, а отсутствие модели — частный случай отсутствия объектов.

# 12 Как читать и проверять многопоточный код

Многопоточные ошибки отличаются от обычных тем, что проявляются *недетерминированно*. Ниже — практические приёмы, применимые к этому проекту.

## 12.1 Правила, по которым устроен код

1. **Один владелец — один поток.** `Queue` принадлежит конкретным потокам и защищена изнутри. Наружу она не отдаёт ссылки на своё содержимое.
2. **Данные между потоками неизменяемы.** `shared_ptr<const Frame>` — компилятор сам не даст записать.
3. **Состояние, разделяемое на чтение, — атомарное.** Счётчики `std::atomic` читаются из HUD без блокировок.
4. **Никаких «флагов на честном слове».** Флаг остановки — `std::atomic<bool>`, а не `bool`.

## 12.2 Типичные ошибки и как их узнать

| Ошибка | Проявление | Как обнаружить |
|---|---|---|
| Гонка данных (data race) | «плавающие» результаты, редкие падения | ThreadSanitizer (`-fsanitize=thread`) |
| Использование после освобождения | падение в случайном месте | AddressSanitizer |
| Дедлок | программа висит на выходе | дамп стеков, `gdb -p` |
| Ложное пробуждение без предиката | падение в `pop()` | проверка предиката в `wait` |
| `joinable` поток при разрушении | `terminate` | порядок флага и `join` |

### 12.3 Как воспроизвести проблему производительности

Приёмы, использованные в проекте:

- **Счётчик вытеснений.** Если `dropped` растёт, обработка не успевает. Это не ошибка, а диагностика.
- **Искусственная перегрузка.** Флаг `-fps 200` у синтетического источника создаёт заведомо избыточный поток кадров: на нём проверяется, что задержка не растёт.
- **Явные метрики в конце прогона.** Первая, средняя и максимальная задержка — три числа, которых достаточно, чтобы отличить рабочую схему от деградирующей.

Проверять надо не «средний FPS», а поведение при перегрузке. Система, которая в среднем быстра, но копит очередь, непригодна для реального времени: задержка в ней растёт линейно и рано или поздно становится недопустимой.

## 13 Библиотеки и фреймворки

### 13.1 OpenCV

Назначение: обработка изображений и видео. Используемые модули:

| Модуль | Что даёт проекту |
|---|---|
| `core` | `cv::Mat`, базовые типы, арифметика |
| `imgproc` | `resize`, `findContours`, `drawContours`, `addWeighted` |
| `imgcodecs` | `imdecode` (JPEG из камеры), `imwrite` |
| `highgui` | `imshow`, `waitKey` |
| `dnn` | `blobFromImage` |
| `video` | `cv::KalmanFilter` |

Модуль `videoio` не используется: видео берётся напрямую из V4L2. Это осознанный выбор — `VideoCapture` скрывает формат и negotiated параметры, а цель проекта — увидеть механику.

### 13.2 ONNX Runtime

Назначение: исполнение моделей в формате ONNX на CPU. Основные понятия:

- `Ort::Env` — окружение (логирование, потоки), одно на процесс;
- `Ort::Session` — загруженная модель;
- `Ort::Value` — тензор; создаётся на уже существующей памяти, поэтому данные не копируются;
- `SessionOptions` — уровень оптимизации графа, число потоков.

Почему `CreateTensor` на памяти `cv::Mat`: входной blob уже лежит непрерывным массивом `float`. Копировать его в тензор — лишние мегабайты на каждый кадр.

### 13.3 Прочие зависимости

| Библиотека | Роль |
|---|---|
| `pthreads` (`Threads::Threads`) | `std::thread`, мьютексы |
| V4L2 (заголовки ядра) | `linux/videodev2.h`, `sys/ioctl.h` |

# 14 Инструменты разработки и отладки

## 14.1 Что искать при падениях

| Симптом | Что это обычно значит |
|---|---|
| `Segmentation fault` | разыменование пустого/висячего указателя, выход за границы массива |
| `terminate called without an active exception` | `joinable std::thread` при разрушении, либо исключение в потоке |
| Зависание на выходе | `join` без снятия флага остановки |
| Рост памяти | утечка `new` без `delete`, цикл ссылок `shared_ptr` |
| «Пустая разметка» | неверный layout выходов, порог уверенности, несовпадение числа классов |

## 14.2 Инструменты

- **AddressSanitizer**: `-fsanitize=address -g`. Указывает строку и тип ошибки памяти. В этом проекте нашёл выход за границы `std::vector<bool>`.
- **UBSan**: `-fsanitize=undefined`. Ловит переполнения знаковых, сдвиги, выравнивание.
- **ThreadSanitizer**: `-fsanitize=thread`. Ищет гонки данных; полезен при изменении очереди.
- **valgrind**: медленнее, но умеет профилировать память.
- **perf**: профилирование CPU.

## 14.3 Проверка без камеры и без весов

```
./scripts/smoke_test.sh
```

Скрипт выполняет пять шагов: сборка, тесты, прогон с моделью-заглушкой, проверка контракта «пустые маски ⇒ половины совпадают», корректная остановка по SIGINT. Модель-заглушка генерируется скриптом `make_stub_model.py`: она имеет те же формы тензоров, что и настоящая, но случайные веса. Этого достаточно для проверки декодера, масок, трекера и отрисовки.

# 15 Разбор найденных ошибок: что и почему ломалось

Раздел полезен тем, что показывает *типичные* ошибки в подобном коде и метод их поиска. Все случаи реальные, из истории разработки проекта.

## 15.1 Выход за границы вектора в трекере

**Симптом.** Тест трекера завершается с `Segmentation fault`.

**Как искали.** Отладчика `gdb` в среде не было. Сборка с `-fsanitize=address` дала точное место: обращение к `std::vector<bool>` по индексу за границей.

**Причина.** Флаги «трек был сопоставлен» создавались по размеру списка треков *до* добавления новых. Затем цикл старения проходил по *новому* размеру, включающему только что добавленные треки.

```
std::vector<bool> track_used(tracks_.size(), false);   //
// ...   : tracks_.push_back(...)
for (std::size_t t = 0; t < tracks_.size(); ++t) {     //  ! UB
    if (!track_used[t]) { ++tracks_[t].missed; }
}
```

**Исправление.** Граница цикла привязана к размеру *флагов*, которые описывают именно «треки, существовавшие до этого кадра».

**Мораль.** Индексная переменная должна ограничиваться тем контейнером, по которому идёт индексация. `operator[]` у `std::vector` не проверяет границы (в отличие от `at()`).

### 15.2 `terminate` при остановке конвейера

**Симптом.** При выходе: `terminate called without an active exception`.

**Причина.** `join()` вызывался до снятия флага остановки. Рабочие циклы крутились по флагу, которого никто не снимал; кроме того, при некотором раскладе рабочий цикл завершался сам, и объект `std::thread` оставался *joinable* — это и вызывает `std::terminate` при разрушении.

**Исправление.** Порядок строго: снять флаг, затем `join`.

### 15.3 Трекер «залипал»: пропущенное уравнение ковариации

Самая содержательная ошибка — она математическая, а не программная. Подробный разбор — в разделе о фильтре Калмана: вызов `kalman.predict()` был заменён на умножение матрицы перехода, из-за чего ковариация ошибки не росла, усиление было близко к нулю, и трек не следовал за объектом.

**Почему не замечалось сразу.** На медленном движении (несколько пикселей за кадр) IoU оставался выше порога, и трек не терялся. Ошибка проявилась бы на реальной камере, где объекты смещаются заметно.

**Мораль.** Тест должен воспроизводить *напряжённый* режим (в данном случае — быстрое движение), иначе он не отличает правильную реализацию от почти правильной.

### 15.4 Нулевая скорость в HUD

**Симптом.** Счётчики кадров в секунду всегда показывали 0,0.

**Причина.** Метрики пересчитывались только внутри ветки обработки полученного результата, а точка отсчёта обновлялась там же. В итоге знаменатель оказывался нулевым или период не накапливался.

**Мораль.** Периодические измерения нужно обновлять независимо от того, пришли ли новые данные; иначе метрика «замерзает» именно в тот момент, когда она важнее всего (поток данных встал).

### 15.5 Отрисовка масок: полный кадр вместо рамки

**Симптом.** На модели, выдающей много объектов (несколько сотен), обработка падает со 30 кадров/с до 6–7, а счётчик вытеснений растёт.

**Что измеряли.** По HUD было видно `objects 300`, то есть дело не в декодере. Замеры по частям:

| Операция (1280×720, 300 объектов) | Время на кадр |
|---|---|
| трекер | 1,7 мс |
| отрисовка без масок | 10,8 мс |
| отрисовка 300 масок (*до* исправления) | 266,5 мс |
| отрисовка 300 масок (*после*) | 17,7 мс |

**Причина.** Маска хранится в размере *всего* кадра, хотя ненулевые пиксели занимают лишь рамку объекта. Код же делал `setTo(color, mask)` и `findContours(mask)` по полному кадру. Стоимость: $300 \times 1280 \times 720 \approx 2{,}8 \cdot 10^8$ операций на кадр — при том что полезная площадь на два порядка меньше.

**Исправление.** Обе операции выполняются над ROI — прямоугольником рамки. Контуры, найденные в ROI, затем сдвигаются на смещение рамки.

```
const cv::Rect roi = instance.box & cv::Rect(0, 0, canvas.cols, canvas.rows);
cv::Mat layer_roi = layer(roi);
layer_roi.setTo(class_color(instance.class_id), instance.mask(roi));
```

`cv::Mat::operator()` не копирует пиксели: ROI — это новый заголовок матрицы, ссылающийся на ту же память, но со своим шагом строки. Поэтому запись в `layer_roi` меняет исходный `layer`. Это идиома «вид на область», и её надо знать: изменение вида изменяет оригинал.

**Мораль.** Сложность алгоритма нужно считать по *реально обрабатываемой* площади, а не по размеру контейнера. И измерять по частям: без замеров легко «оптимизировать» не то место — трекер, который был подозреваемым номером один, занимал менее 2% времени.

### 15.6 Молчаливая потеря объектов при неожиданном выходе модели

**Симптом.** При нестандартной форме второго выхода модели на экране нет ни объектов, ни сообщений об ошибке.

**Причина.** Результат заполнялся *только* внутри ветки «построить маски». Если второй выход не был картой прототипов, функция возвращала пустой список.

**Исправление.** Явная проверка формы выхода и *деградация*: при отсутствии прототипов отдаются хотя бы рамки, а в лог пишется предупреждение.

**Мораль.** Любая «деградация» должна быть видимой. Пустой результат без сообщения — худший вид ошибки: он выглядит как корректная работа на кадре без объектов.

## 16 Как расширять проект

Разделение на интерфейсы выбрано так, чтобы типичные изменения затрагивали один файл.

### 16.1 Добавить новый источник кадров

Достаточно реализовать `Source` и выбрать реализацию в `main`:

```
class FileSource final : public Source {
public:
    bool open(std::string& error) override;
    bool grab(FramePtr& out, std::string& error) override;
    int width() const override;
    int height() const override;
};
```

Конвейер, трекер и отрисовка при этом не меняются: они работают с `Source&`.

### 16.2 Поддержать другой формат выходов модели

Добавляется новая ветка декодирования и новое значение параметра `out_layout`. Важно, что формат задаётся *явно*: пользователь обязан указать, какой у него экспорт, а программа печатает фактические формы тензоров при загрузке — расхождение видно сразу.

### 16.3 Улучшить сопоставление в трекере

Жадный алгоритм локализован в одной функции. Замены по возрастанию сложности:

1. венгерский алгоритм (оптимальное назначение по суммарному IoU);
2. каскадное сопоставление: сначала по IoU, затем по расстоянию между центрами;
3. учёт класса и цвета/гистограмм;

4. DeepSORT с эмбеддингами внешнего вида.

Все они вставляются на место шага сопоставления и не затрагивают очередь, потоки или отрисовку.

### 16.4 Что менять не следует

- **Политику переполнения очереди** — вытеснение старого является смыслом схемы реального времени.
- **Неизменяемость кадра** на границе потоков: именно она обеспечивает потокобезопасность без блокировок.
- **Явность форматов**: любой автоподбор скрывает причину ошибок.

## 17 Итог: принципы, которые переносятся на другие проекты

1. **Разделяй стадии по времени отклика.** Захват (жёсткий темп) и обработка (переменное время) — разные потоки.
2. **Ограничивай буферы и выбирай политику переполнения осознанно.** Для реального времени правильная политика — вытеснение старого.
3. **Делай данные неизменяемыми на границе потоков.** `shared_ptr<const T>` убирает гонки без блокировок.
4. **Никогда не блокируй производителя.** Иначе камера встанет.
5. **Задавай форматы явно.** Автодетект скрывает ошибки.
6. **Пиши тест на каждую найденную ошибку.** Тест на движение в трекере и тест баланса очереди появились именно так.
7. **Переводи исключения в коды возврата на границе потоков.** Иначе `std::terminate`.
8. **Измеряй задержку, а не только пропускную способность.** Система может быть быстрой в среднем и непригодной в реальном времени.

## A Сводка формул

$$\text{letterbox:}\quad s = \min(S/W, S/H), \quad x = su + p_x \tag{7}$$

$$\text{обратно:}\quad u = (x - p_x)/s \tag{8}$$

$$\text{IoU:}\quad \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \tag{9}$$

$$\text{сигмоида:}\quad \sigma(z) = \frac{1}{1 + e^{-z}} \tag{10}$$

$$\text{порог маски:}\quad \sigma(z) > t \iff z > \ln \frac{t}{1 - t} \tag{11}$$

$$\text{Калман, предсказание:}\quad \hat{x}^- = F\hat{x}, \quad P^- = FPF^\top + Q \tag{12}$$

$$\text{Калман, коррекция:}\quad K = P^- H^\top (HP^- H^\top + R)^{-1} \tag{13}$$

$$\hat{x} = \hat{x}^- + K(z - H\hat{x}^-), \quad P = (I - KH)P^- \tag{14}$$

$$\text{Precision/Recall:}\quad \frac{TP}{TP + FP}, \quad \frac{TP}{TP + FN} \tag{15}$$

$$\text{mAP:}\quad \frac{1}{n_c} \sum_c \mathrm{AP}_c \tag{16}$$

## B Словарь терминов

| | |
|---|---|
| **blob** | непрерывный массив чисел, здесь — вход сети в формате NCHW. |
| **letterbox** | вписывание изображения в квадрат с сохранением пропорций и полями. |
| **NMS** | non-maximum suppression, подавление перекрывающихся детекций. |
| **IoU** | intersection over union, мера перекрытия двух областей. |

**anchor** позиция на карте признаков, для которой сеть предсказывает объект.

**прототип маски** одна из 32 базовых масок, линейная комбинация которых даёт маску объекта.

**RAII** привязка владения ресурсом к времени жизни объекта.

**pimpl** идиома сокрытия деталей реализации за неполным типом.

**вытеснение** удаление самого старого элемента при переполнении очереди.

**rpath** путь поиска динамических библиотек, вшитый в исполняемый файл.